Advanced Kanban Board Assignment

📌 Assignment Overview

Build a purely client-side Kanban board with:

✅ Drag-and-drop task management

✅ Role-based access control (3 distinct roles)

✅ Advanced Realtime notification system (Use **Event Emitter/Bus**) design pattern)

✅ Web Worker-powered data exports

✅ Supabase auth integration (session management only)

✅ Use **IndexedDB** for data persistence, preferably dexie.js wrapper or choose from all

📌 Core Requirements

1. 📋Task Management

a. **Task Fields :**

i. `Title` (Required, max 100 chars)
ii. `Description` (Markdown supported)
iii. `Assignee` (Dropdown of authenticated users)
iv. `Due Date` (Future dates only, timezone-aware, date time picker, date-fns)
v. `Priority` (Low/Medium/High/Critical)
vi. `Status` (Todo/In Progress/ UAT / Done)
vii. `Comments` (Timestamps, edit history)
viii. `Attachments` (find a way to save images in indexdb)

b. **Task Actions:**

i. Drag-and-drop between columns (state machine to handle states)
ii. Double-click to edit in a modal
iii. Persist open modals on refresh

c. **State Preservation**

i. Remains open through page refreshes
ii. Restores scroll position in comments
iii. Remembers last active tab (details/comments/attachments)

d. **Multi-Task Viewing**
    i. Allow opening multiple modals (stacked/z-index)
    ii. Keyboard shortcuts to cycle through open modals

2. 📢 Comprehensive Notification System

a. Global System-Wide Notifications (Default for All Tasks)

i. Task Due Reminders

1. Configurable lead times (e.g., 1 day, 1 hour, 15 minutes before due)
2. Escalation path for overdue tasks (notify manager after X hours)
3. Multiple reminders per task

ii. Status Change Alerts (`When a task moves between columns (Todo → In Progress → Done)`

iii. Assignment Notifications (When a user is assigned/reassigned a task)

iv. Comment Activity

1. Notify the task assignee of new comments
2. @mention support for specific users

v. Settings

1. Enable/disable per notification type
2. Default timing for reminders
3. Preferred channels (in-app toast, browser notification)

b. Per-Task Notification Overrides

i. Disable specific notifications for a task
ii. Adjust reminder times (e.g., "Notify me 10 minutes before due")

3. Role-Based Access Control (RBAC)

Roles & Permissions

Action	Admin	QA	Developer
Create/Delete Tasks	✅	❌	❌
Edit All Fields	✅	❌	❌
Reassign Tasks	✅	❌	❌
Move to "Done"	✅	✅	❌
Add Comments	✅	✅	✅
Configure Notifications	✅	❌	❌

Design

Use google’s material web components or tailwind

Guidelines

Suggested Implementation Approach

1. Phase 1: Base Kanban with localStorage
2. Phase 2: Auth integration (Supabase)
3. Phase 3: Notification system
4. Phase 4: Web Worker exports
5. Phase 5: RBAC enforcement
6. Phase 6: Polish and edge cases

Modular Architecture

```
src/
├── core/               # Framework-agnostic logic
│   ├── auth/           # Supabase auth adapters
│   ├── notifications/  # Notification strategies
├── domain/             # Business logic
│   ├── tasks/          # Task entity + use cases
│   ├── roles/          # RBAC policies
├── infrastructure/     # Client-side storage
│   ├── localStorage/   # Task repository
│   ├── cache/          # Session management
└── presentation/       # UI Layer
    ├── components/     # Design system
    ├── views/          # Page-level components
```

🔍 Code Review Checklist

For Every Pull Request:

1. SOLID:
   - No class > 300 lines
   - Interfaces for cross-module communication
2. DRY:
   - Shared utility functions used

    - No duplicate CSS/validation logic
3. Design System:
    - Uses centralised spacing vars
    - Follows typography scale
4. Performance:
    - Web Worker for heavy ops
    - Debounced event handlers

Security Constraints

1. Client-Side Enforcement
    - All RBAC checks must happen in UI logic
    - Defensive coding against manual localStorage tampering
    - Read-only views for unauthorised actions
2. Data Exposure Prevention
    - Never expose other users' email addresses
    - Sanitise all comment input
    - Prevent HTML/script injection in any field
3. Session Protection
    - Auto-logout after 2 hours of inactivity
    - Clear sensitive data on logout

📝 Submission Requirements

1. **Code Repository**
    - Clean commit history
    - `.gitignore` proper configuration
    - ESLint/Prettier setup
2. **Documentation**
    - Setup instructions
    - Architecture decisions
    - Known limitations
3. **Demo Video (2-3 mins)**
    - Show all role flows
    - Demonstrate exports with 10k+ tasks